

[問題23]

下図のような電車車両入替え用の線路があります。この線路は、図の下側は行き止まりになっており、上側の左右それぞれの線路は、左が入り口専用、右が出口専用になっています。ここを走る車両は10両あり、それぞれ1から10までの番号が付けられています。

この線路の左上から車両が入って来て行き止まりの線路に入ることを車両の番号で表し、また行き止まりに入っている先頭の車両が出て行くことを0で表わすと、これを並べて車両の出入りの様子を表わすことができます。例えば

1 6 0 8 10

は、「まず1番が入り、次いで6番が入り、次に先頭の1台(今入ってきた6番)が出てゆき、8番が入り、10番が入る」という順序の出入りを示しています(右下図参照)。

この形式の数字の列を入力データとし、出て行く車両の番号をその順番に出力するプログラムを、[問題22](2)で作ったスタックライブラリを用いて作成してください。

なお、最初に行き止まりの線路に車両は入っていないものとします。また、データは常に車両の出入りを正しく表すものが与えられます。例えば、行き止まりの線路が空なのに次の入力「0」である、などという間違ったデータは入力されないものとして構いません。但し、入力終了時に線路が空になっているとは限りません。したがって、入力の終りはEOFで判定する必要がありますことに注意してください(入力には、scanf() 関数を使うのが簡単です。また、下の実行例のテスト用入力データは /home/pub/sample/prog23.datにあります)。

(prog 23. c)

入力

入ってくる車両の番号(整数)

入ってくる車両の番号または0(整数)

...

入ってくる車両の番号または0(整数)

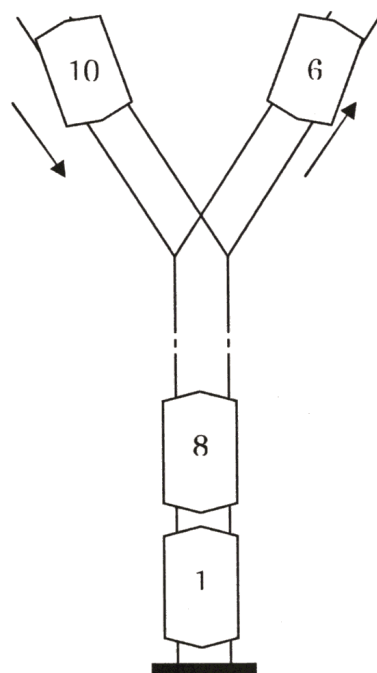
出力

出て行く車両の番号

入力例	出力例
1	6
6	10
0	8
8	1
10	
0	
0	
0	

サンプルデータでの実行方法

```
$ a.out < /home/pub/sample/prog23.dat
```



[問題24]

次のプログラムを段階的詳細化およびデータ抽象化の考え方に基づいて作りなさい。
但し、[問題22](2)で作ったスタックライブラリを必ず用いること。

よく切った53枚のトランプ(ジョーカーを含む)から10枚のカードの山を2つ作り、それぞれの山を2人に配ります(よく切ったトランプは[補足]に示した乱数を使って作ります)。

次に、2人が同時に自分の山の1番上から1枚ずつカードを出します。その時、カードの数字の大きい方が勝ち、小さい方が負け、同じ場合は引き分けとし、勝った場合には2点、引き分けの場合は1点、負けた場合は0点を加算するものとします。但し、ジョーカーはどの数字よりも強いものとします。

すべての山がなくなった時点で、ゲームの終了とします。各回のカードとその勝敗、および2人の最終的な得点を表示しなさい。

(prog24.c)

(実行例)

```
( 1回目) Player1: D[ 3] < Player2: S[ 7]   総得点 - Player1:  0, Player2:  2
( 2回目) Player1: D[ 4] > Player2: C[ 3]   総得点 - Player1:  2, Player2:  2
( 3回目) Player1: H[ 7] < Player2: H[12]   総得点 - Player1:  2, Player2:  4
( 4回目) Player1: C[ 1] = Player2: D[ 1]   総得点 - Player1:  3, Player2:  5
( 5回目) Player1: S[ 6] > Player2: H[ 5]   総得点 - Player1:  5, Player2:  5
( 6回目) Player1: D[ 5] > Player2: S[ 4]   総得点 - Player1:  7, Player2:  5
( 7回目) Player1: D[ 8] > Player2: D[ 6]   総得点 - Player1:  9, Player2:  5
( 8回目) Player1: J      > Player2: D[ 2]   総得点 - Player1: 11, Player2:  5
( 9回目) Player1: S[ 2] < Player2: H[ 3]   総得点 - Player1: 11, Player2:  7
(10回目) Player1: H[ 2] < Player2: H[ 4]   総得点 - Player1: 11, Player2:  9
```

Player1の得点: 11

Player2の得点: 9

レポート

次のものをE-mailで提出しなさい。

- ・ヘッダファイル stack.h とライブラリのソースファイル stack.c (添付ファイル)
- ・[問題23, 24]のプログラム prog23.c, prog24.c (添付ファイル)と
実行結果(本文)
- (注意) [問題24]は、段階的詳細化の過程が分かるようにしておくこと。
- ・演習の感想(本文)

※ 再提出の際も stack.h と stack.c を添付するのを忘れないこと

(提出先) E-mail: k-kenta@hamako-ths.ed.jp , 件名: プロ技演習23～24
(提出期限) / ()

[補足] 乱数の発生方法

例えば、次のようなプログラムで、1から100の間の整数乱数を発生することができます。

```
1: #include <stdio.h>
2: #include <stdlib.h> ... ①
3: #include <time.h> ... ②
4:
5: main()
6: {
7:     int rnd;                // 発生した乱数を格納する変数
8:
9:     srand(time(NULL));      // 乱数系列の変更 ... ③
10:
11:     while ( ... ) {
12:         rnd = rand() % 100 + 1; // 1～100の間の乱数を1つ発生 ... ④
13:         printf("%d\n", rnd);    // 発生した乱数を表示
14:     }
15: }
```

(プログラムの説明)

ポイントとなるのは、①～④の文です

12行目

- rand()関数は、0 ～ RAND_MAX の範囲の整数乱数を返します。
- RAND_MAX の値は、stdlib.h に定義されており、本校で使用しているコンパイラ (gcc-8.4.1) では、2147483647 です。
- 1 ～ 100 の乱数を作るために、rand()の戻り値を 100 で割った余りを求めています。余りは 0 ～ 99 の乱数となるため、これに 1 を加えることで 1 ～ 100 の乱数になります。

9行目

- rand()関数で発生する乱数は疑似乱数といって、実体は計算で作り出している数列です。そのため、rand()関数は常に同じ乱数を発生します。
- 常に同じ乱数ではほとんど役に立たないので、作り出す数列の発生系列を実行するたびに変更してやる必要があります。
- srand()は、発生させる疑似乱数の系列を指定する関数です。
引数は unsigned int で、引数の値が同じなら、rand()関数は同じ疑似乱数を発生させます。逆に言えば、srand()の引数を変更してやることで、別系列の疑似乱数を発生させることができます。
- そこで、srand()の引数に time()関数で得られる現在時刻を利用することで、実行するたびに乱数の発生系列を変更するようにしています。
- time(NULL)は、グリニッジ標準時の 1970 年 1 月 1 日 00:00:00 から、現在までの経過時間 (秒) を返します。

2行目

- stdlib.h は、rand(), srand() 関数を利用するのに必要なヘッダファイルです。

3行目

- time.h は、time() 関数を利用するのに必要なヘッダファイルです。